



汇编语言

5 传送类指令与算术运算类指令

张青

ieqzhang@zzu.edu.cn

教学内容

- 复习寻址方式

教材章节：3.5

慕课：4.5

- 传送类指令

- 算术运算类指令

教材章节：4.2,4.3

慕课：5

- 位操作类指令导学

教材章节：4.4

慕课：6

- ▶ 识别寻址方式

- ▶ MOV指令

- ▶ XCHG、LEA指令

- ▶ PUSH和POP指令

- ▶ 状态标志

- ▶ 算术运算指令

- ▶ 常见语法错误

- ▶ 位操作指令

学习目标



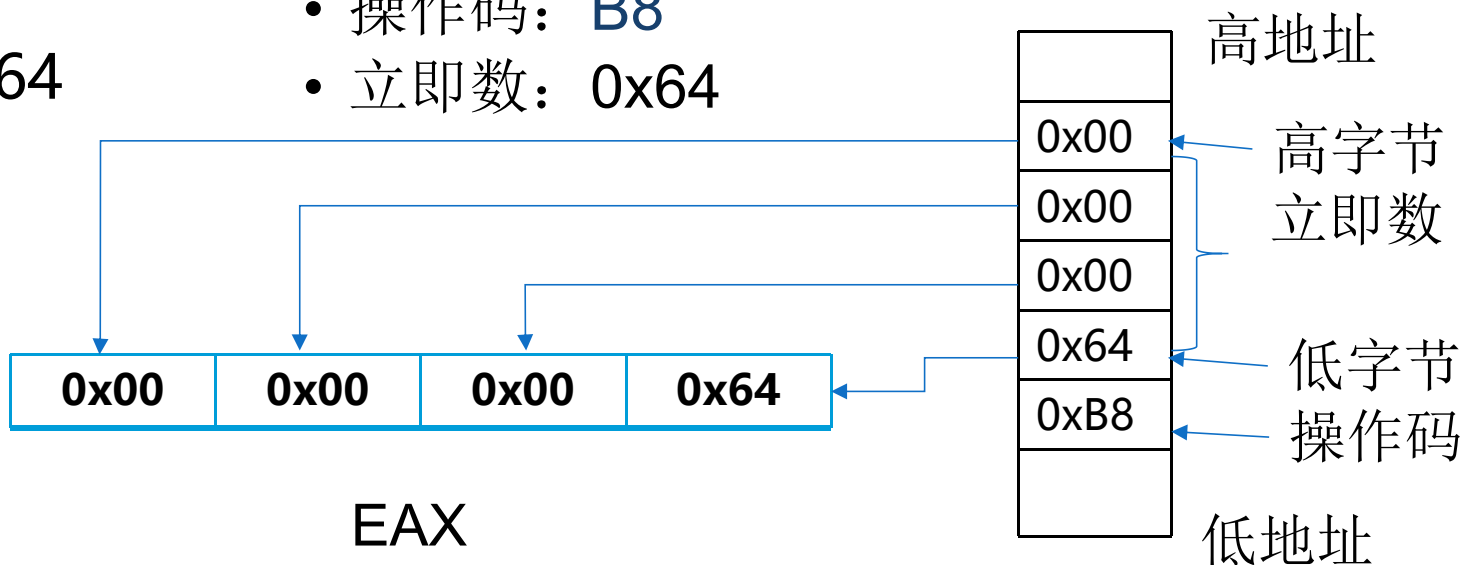
- 能正确书写数据传送和算术运算指令
- 能判断指令执行结果对标志位的影响
- 能编写指令序列完成数据传送和算术运算
- 能写出C语言程序对应的汇编指令
- 能写出汇编指令序列对应的C语句
- 能用GDB调试程序
- 养成细致严谨的态度，注意C语言与汇编语言之间的关系，培养系统观

数据寻址方式

- 指令有两部分：操作码和操作数
 - 操作码：处理器要执行哪种操作
不可缺少，用助记符表示
 - 操作数：指令执行的参与者
各种操作的对象，需要通过地址指示
- 数据寻址方式：通过地址查找数据（操作数）
 - 立即数寻址：数据在指令代码中，用常量表达
 - 寄存器寻址：数据在寄存器中，用寄存器名表示
 - 存储器寻址：数据在主存中，用存储器地址代表

立即数寻址

- 操作数紧跟操作码，是机器代码的一部分
- 操作数从指令代码中立即得到，即立即数 (Immediate) ，用常量形式直接表达
- 立即数寻址方式只用于源操作数，常用来给寄存器和存储单元赋值
- 例如：`mov eax, 100`
 - 机器代码：B8 64 00 00 00
 - 操作码：B8
 - 立即数：0x00000064
- `mov al, 100`
 - 机器代码：B8 64
 - 操作码：B8
 - 立即数：0x64



寄存器寻址

- 操作数存放在处理器的内部寄存器中
- 用寄存器名表示它的内容
- 绝大多数指令采用通用寄存器寻址
- 部分指令支持专用寄存器，例如段寄存器
- 寄存器寻址方式简单快捷，最常使用
- 例如：MOV EBX,EAX

64位通用寄存器：RAX RBX RCX RDX

32位通用寄存器：EAX EBX ECX EDX

16位通用寄存器：AX BX CX DX

8位通用寄存器：AH AL BH BL

- 操作数在主存中，通过存储器地址指示
- 编程时，存储器地址使用包含段选择器和偏移地址的逻辑地址
 - 段选择器（段寄存器）指示段基地址
 - 默认规定：数据在DS指向的数据段；EBP或ESP作为基地址，数据在SS指向的堆栈段
 - 显式说明：使用段超越指令前缀，段寄存器名后跟英文冒号
 - 偏移地址由各种寻址方式计算
 - 常被称为**有效地址EA**（Effective Address）

偏移地址的组成

64位有效地址 = 基址寄存器 + (变址寄存器 × 比例) + 位移量

- 基址寄存器：任何一个64位通用寄存器之一，RIP
- 变址寄存器：除RSP外的任何64位通用寄存器之一
- 比例：1, 2, 4, 8或16
- 位移量：8, 16, 32或64位有符号值



变化出多种主存寻址方式

1. 直接寻址

- 有效地址只有位移量部分，直接包含在指令代码中
- 常用于存取变量
- 仅适用于IA-32处理器保护模式
- 64位模式下不使用这种寻址方式
- 例如：在IA-32处理保护模式下

MOV ECX,[COUNT] #COUNT是变量

MOV ECX, DS:[405000H]

- 指令代码：8B 0D 00 50 40 00
- 操作码和寻址方式：8B 0D
- 操作数：有效地址 00405000H

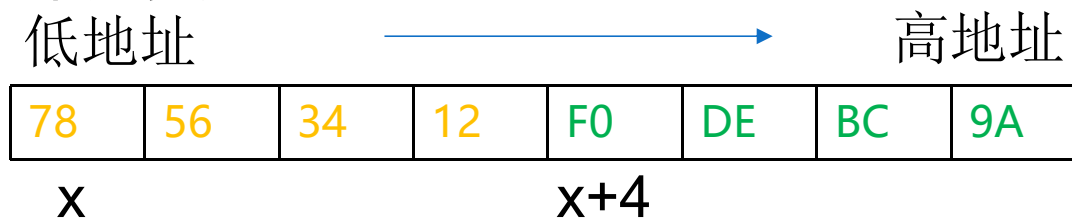
2.RIP相对寻址方式

- 64位模式下最常使用的一种存储器寻址方式
- 有效地址 = RIP+64位位移量

行数	线性地址	机器码	汇编语句
			.data
9	0000	78563412	x: .long 0x12345678,0x9ABCDEF0
9		F0DEBC9A	
10	0008	31323334	y: .ascii "12345"
10		35	
			.text
14	0000	8B050000	mov eax,x[rip] /*有效地址=x+rip,eax=0x12345678 */
14		0000	
17	0010	8B050800	mov eax,y[rip] /*有效地址=y+rip,eax=0x34333231 */
17		0000	
20	0020	488B0504	mov rax,x[rip+4] /*有效地址=x+rip+4,rax=0x343332319ABCDEF0 */
20		000000	

3. 寄存器间接寻址

- 有效地址存放在寄存器中(寄存器内容 = 偏移地址 = 有效地址)
- 用中括号括起寄存器
- 可以方便地对数组的元素或字符串的字符进行操作
- 寄存器间接寻址没有说明存储单元类型
- 例如:



```
.data
x: .long 0x12345678,0x9ABCDEF0
.text
lea rax, x[rip] /*x地址复制到rax*/
mov bl, [rax] /*有效地址=rax, bl=0x78 */
mov bx, [rax] /*有效地址=rax, bx=0x5678 */
mov ebx,[rax] /*有效地址=rax, ebx=0x12345678 */
mov rbx,[rax] /*有效地址=rax, rax=0x9ABCDEF012345678 */
```

4. 寄存器相对寻址

- 有效地址是寄存器内容与位移量之和
- 可以方便地对数组的元素或字符串的字符进行操作
- 例如:

```
.data  
x: .long 0x12345678, 0x9ABCDEF0
```

```
.text
```

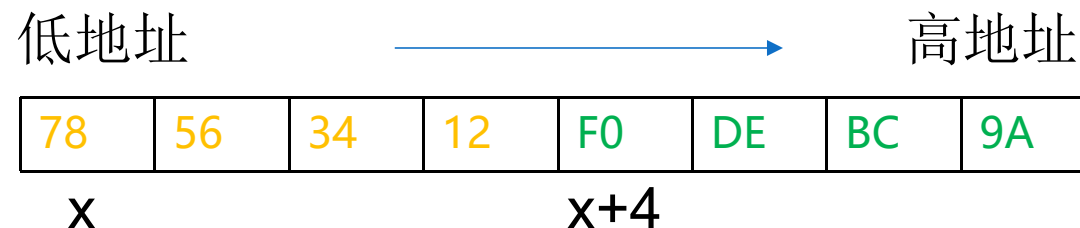
```
lea rax, x[rip] /*x地址复制到rax*/
```

```
mov bl, [rax+1] /*有效地址=rax+1, bl=0x56 */
```

```
mov bx, [rax+2] /*有效地址=rax+2, bx=0x1234 */
```

```
mov ebx, [rax+4] /*有效地址=rax+3, ebx =0x9ABCDEF0 */
```

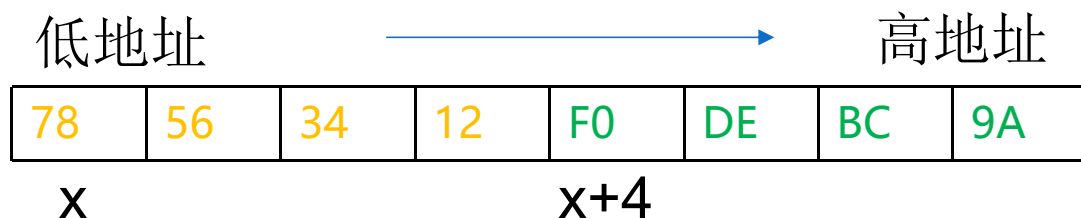
```
mov rbx, [rax+4] /*有效地址=rax+4, rbx =0x000000009ABCDEF0*/
```



主存以字节为可寻址单位
地址的加减是以字节为单位

5. 基址变址寻址

- 使用变址寄存器寻址操作数
- 便于支持二维数组等数据结构
- 例如：

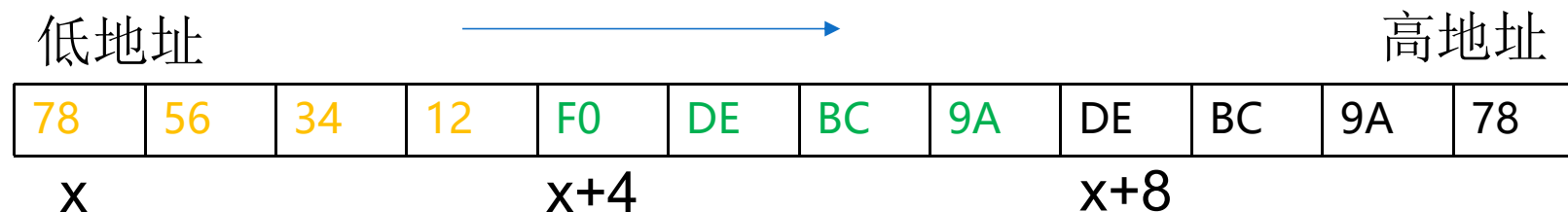


```
.data
x: .long 0x12345678, 0x9ABCDEF0
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 0
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x12345678 */
mov rbx, 4
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x9ABCDEF0 */
```

6.相对基址变址寻址

- 有效地址由基址寄存器的内容加上变址寄存器内容再加上一个64位的位移量构成

• **有效地址 = 基址寄存器 + 变址寄存器 + 64位位移量**



.data

x: .long 0x12345678, 0x9ABCDEF0, 0x789ABCDE

.text

lea rax, x[rip] /*x地址复制到rax*/

mov rbx, 4

mov eax, [rax+rbx+2] /*有效地址=rax+rbx+2, eax = 0xBCDE9ABC */

mov eax, [rax+rbx+4] /*有效地址=rax+rbx+4, eax = 0x789ABCDE */

7. 带比例的基址变址寻址

- 变址寄存器内容乘以比例1（可省略），2，4或8的变址寻址
- 比例1、2、4和8对应8、16、32和64位数据的字节个数，便于以数组元素为单位寻址相应数据

- 例如：

78	56	34	12	F0	DE	BC	9A	9A	78	56	34
----	----	----	----	----	----	----	----	----	----	----	----

低地址 → 高地址

x x+4 x+8

```
.data
x: .long 0x12345678, 0x9ABCDEF0, 0x3456789A
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 1
mov eax,[rax+rbx*4] /*有效地址=rax+rbx+2, eax = 0x9ABCDEF0 */
mov eax,[ rax+rbx*4+4] /*有效地址=rax+rbx+4, eax = 0x3456789A */
```

例3-9 数组访问

```
//eg0309.c
#include <stdio.h>
int array[]={12345,-12345};
int i,*pi=array;
int main(){
    printf("%d\n",array[0]);
    i=1;
    printf("%d\n",array[i]);
    i=2;
    printf("%d\n",array[i]);
    printf("%d\n",*pi);
    pi++;
    printf("%d\n",*pi);
    pi++;
    printf("%d\n",*pi);
    return 0;
}
```

mov	edx, DWORD PTR array[rip]	#RIP相对寻址, array[0]
mov	edx, DWORD PTR array[rip+4]	#RIP相对寻址, array[1]
mov	edx, DWORD PTR array[rip+8]	#RIP相对寻址, array[2]
mov	rax, QWORD PTR pi[rip]	/*rax=pi*/
mov	edx, DWORD PTR [rax]	/*寄存器间接寻址, *pi */
mov	rax, QWORD PTR pi[rip]	
lea	rdx, 4[rax]	#rdx=pi+4
mov	QWORD PTR pi[rip], rdx	
mov	edx, DWORD PTR 4[rax]	#寄存器相对寻址, *(pi+1)
mov	rax, QWORD PTR pi[rip]	
lea	rdx, 4[rax]	#rax=pi+8
mov	QWORD PTR pi[rip], rdx	
mov	edx, DWORD PTR 4[rax]	#寄存器相对寻址, *(pi+2)

讨论：源操作数的寻址方式是什么？



//addressing.c

```
....  
int main(){  
    int i;  
    char *p;  
    f=1.0;  
    for(i=0;i<5;i++)  
        array[i]+=10;  
    for(p=str;*p;p++)  
        *p+=1;  
    return 0;  
}
```

mov DWORD PTR f[rip], 0x3f800000

立即数寻址

.....
mov eax, 0

立即数寻址

.....
lea rcx, array[rip]

RIP相对寻址

movsx r8, eax

寄存器寻址

mov edx, DWORD PTR [rcx+r8*4]

带比例基址变址寻址

add edx, 10

立即数寻址

.....

lea rdx, str[rip]

RIP相对寻址

add eax, 1

立即数寻址

mov BYTE PTR [rdx], al

寄存器寻址

add rdx, 1

立即数寻址

movzx eax, BYTE PTR [rdx]

寄存器间接寻址

....

gcc -S -Og -masm=intel -c -o addressing.S addressing.c

寻址的数据是什么?



dvar: .long 0x12345678,12

```
lea rbx, dvar[rip]
mov eax, [rbx]
mov ecx, dvar[rip]
mov rsi, 2
mov edi, [rbx+rsi*2]
mov edx, [rbx+rsi+2]
mov al, byte ptr dvar[rip+2]
```

RBX = 0x00405000

EAX = 0x12345678

ECX = 0x12345678

RSI = 2

EDI = 0x0000000C

EDX = 0x0000000C

AL = 0x34

dvar → 0x00405000

→ 0x00405004

+4

地址

00

00

00

0C

12

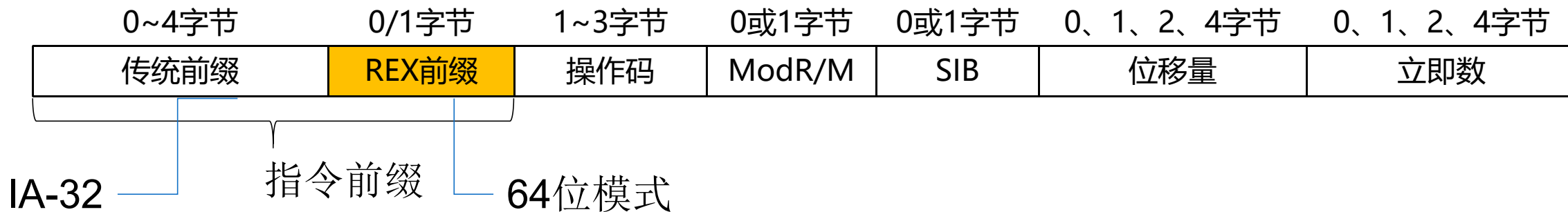
34

56

78

X86处理器指令格式 (Instruction Format)

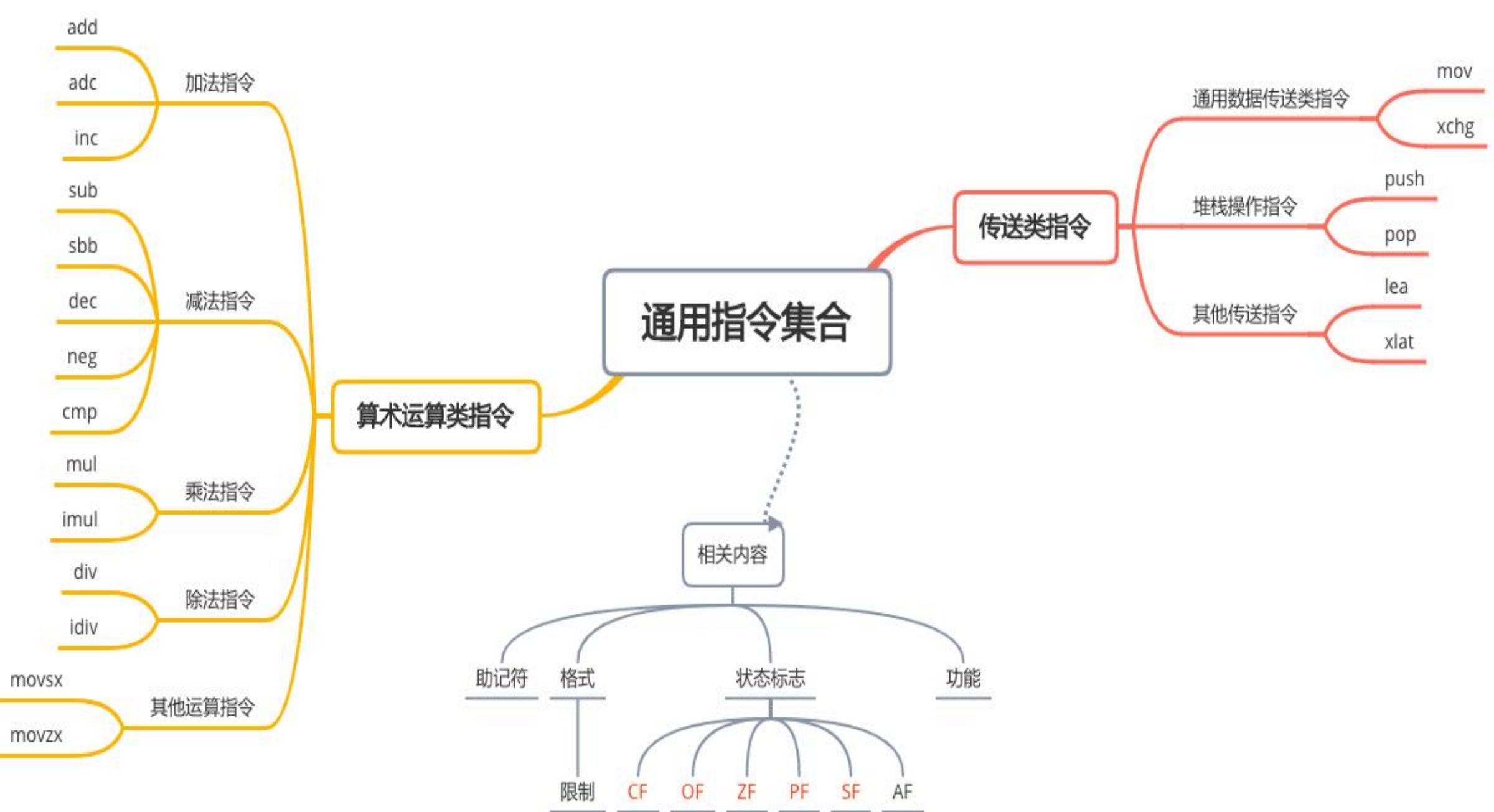
- x86处理器采用可变长度指令格式 (复杂指令集合)
- 操作码
 - 可选的指令前缀 (用于扩展指令功能)
 - 1 ~ 3字节的主要操作码
- 操作数
 - 可选的寻址方式域 (包括ModR/M和SIB字段)
 - 可选的位移量
 - 可选的立即数





学习指令的注意事项

- **指令的功能**——该指令能够实现何种操作。通常指令助记符就是指令功能的英文单词或其缩写形式
- **指令支持的寻址方式**——该指令中的操作数可以采用何种寻址方式
- **指令对标志的影响**——该指令执行后是否对各个标志位有影响，以及如何影响
- **其他方面**——该指令其他需要特别注意的地方，如指令执行时的约定设置、必须预置的参数、隐含使用的寄存器等



数据传送类指令

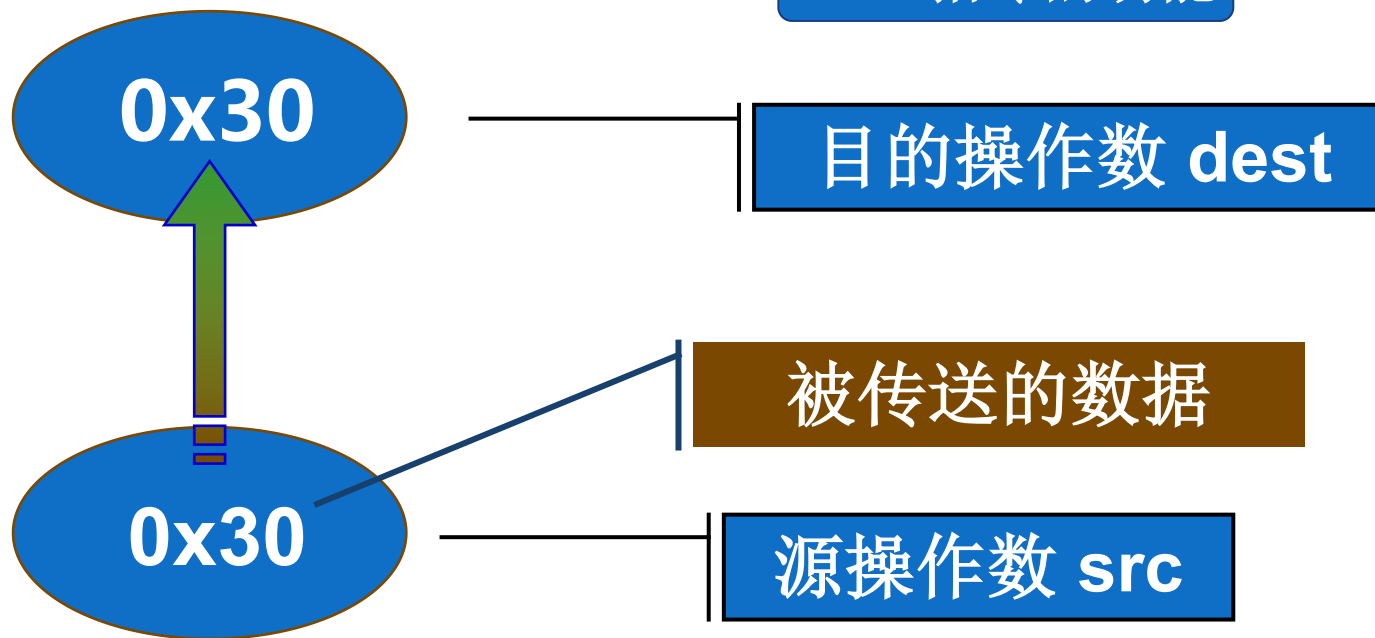
- 数据传送
 - 把数据从一个位置传送到另一个位置
 - 计算机中最基本的操作
 - 程序设计中最常使用的指令
- 除标志寄存器传送指令外，均不影响标志位

全面而准确地理解每条指令的功能和应用是编写汇编语言程序的关键也是理解处理器如何进行数据处理的核心

通用数据传送指令

- 提供方便灵活的通用数据传送操作
- 主要有传送MOV和交换XCHG指令
- 地址传送指令LEA
- 堆栈操作指令PUSH, POP

MOV指令的功能



传送指令MOV



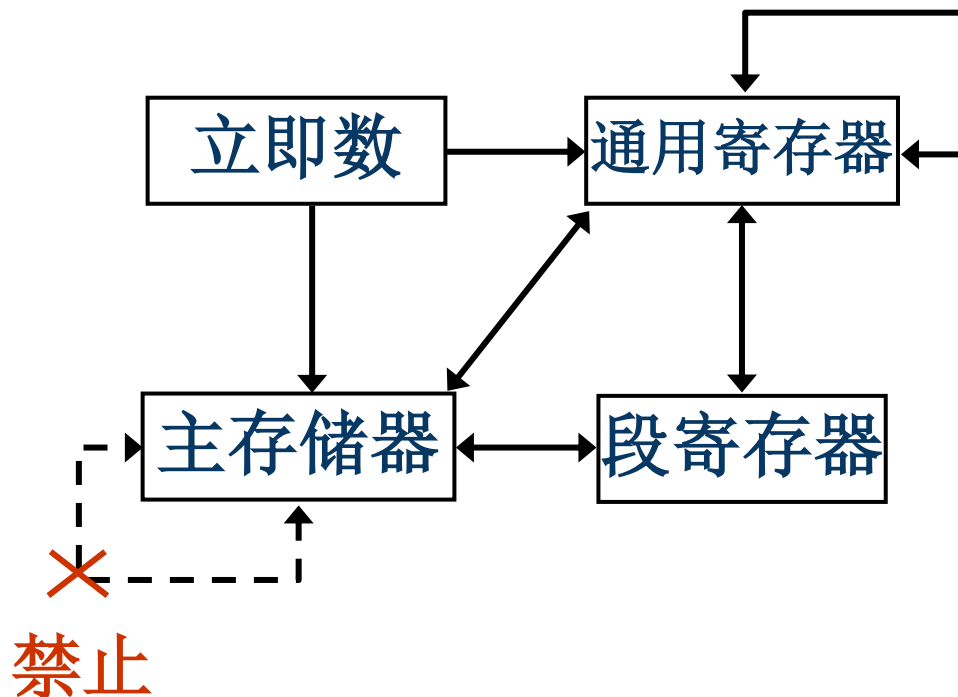
- 把一个字节、字、双字、四字的操作数从源位置传送至目的位置

MOV reg/mem,imm

MOV reg/mem/seg,reg

MOV reg/seg,mem

MOV r16/m16,seg



〔例4-1〕：C语言中的赋值语句



```
/* eg0401.c*/  
long exchange(long *xp, long y)  
{  
    long x = *xp;  
    *xp = y;  
    return x;  
}
```



xp: rcx, x: eax, y: edx

```
mov    eax, DWORD PTR [rcx]  
mov    DWORD PTR [rcx], edx
```

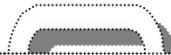
交换指令XCHG

- 将源操作数和目的操作数内容交换

XCHG reg,reg/mem

XCHG reg/mem,reg

- 通用寄存器与通用寄存器之间
 - 通用寄存器或存储器之间
- 空操作指令NOP (= XCHG EAX,EAX)
 - 处理器执行空操作该指令，需要花费时间，在主存中占用一个字节空间
 - 机器码：90
 - 实现短时间延时
 - 临时占用代码空间



```
xchg esi,edi  
xchg esi,[rdi]  
xchg al, var[rip]  
xchg rax,var[rip]
```

交换指令示例-大小端转换



.data

x: .long 0x23430a7b

.text

```
mov al, byte ptr x[rip]
xchg al, byte ptr x[rip+3]
xchg byte ptr x[rip], al
mov al, byte ptr x[rip+1]
xchg al, byte ptr x[rip+2]
xchg al, byte ptr x[rip+1]
```

23	43	0a	7b
x	x+1	x+2	x+3

#取第1个字节

#与第4个字节交换

#实现低1、4个字节互换

#同上，AL=第2个字节

#与第3个字节交换，AL=第3个字节

#实现第2、3个字节互换

- 拼写错误、多余的空格、不正确的标点、太过复杂的常量或表达式
- 操作数类型不匹配、无明确类型、错用（段）寄存器、两个存储器操作数、超出数据范围.....

注意1：双操作数必须类型一致

<code>mov esi,dl</code>	#错误：类型不一致 #esi为32位寄存器，dl为8位寄存器
<code>mov esi,edx</code>	#正确：两个32位寄存器传送
<code>mov al,0x050a</code>	#错误：类型不一致 #0x050a超过了寄存器AL范围
<code>mov eax,#050a</code>	#正确：双字量数据传送

注意2：操作数必须有明确的类型

mov [rbx], 255 #错误：无明确类型

mov byte ptr [rbx],255 #正确：BYTE PTR说明是字节操作

mov word ptr [ebx],255 #正确：WORD PTR说明是字操作

mov dword ptr [ebx],255 #正确：DWORD PTR说明是双字操作

注意3：双操作数不允许都是主存单元



#假设dbuf1和dbuf2是两个双字变量

`mov dbuf2[rip],dbuf1[rip]` #错误：两个操作数都是存储单元

`mov eax,dbuf1[rip]` #正确：`eax = dbuf1`（将dbuf1内容送eax）

`mov dbuf2[rip],eax` #正确：`dbuf2 = eax`（将eax内容送dbuf2）

注意4：不可随意操作专用寄存器

mov ds,100

#错误：立即数不能直接传送段寄存器

mov ax,cs

mov ds,ax

#正确：通过AX间接传送给DS

注意5：新字节寄存器的用法



- 不能同时使用传统的高字节和新的字节寄存器

<code>mov dil, al</code>	#正确，新字节寄存器和传统低字节寄存器
<code>mov dil, ah</code>	#错误：不能同时使用 <code>ah</code> 和新增字节寄存器
<code>mov bl, ah</code>	#正确，两个传统字节寄存器
<code>mov dil, r8b</code>	#正确，两个新字节寄存器

课堂练习：下面指令是否正确

- 1 `mov dvar[rip+4],dvar[rip]`
- 2 `mov edi,si`
- 3 `mov eax,0x50a`
- 4 `mov al,0x50a`
- 5 `mov dword ptr [rbx],255`
- 6 `mov [rbx+4],200`
- 7 `mov ds, 100`
- 8 `mov ax, ds`
- 9 `mov edx, [ebx+0xff]`
- 10 `mov [rsi], dword ptr [rdi]`
- 11 `mov ecx, cs`
- 12 `xchg al, byte ptr dvar[rip+3]`
- 13 `xchg 100, al`

- | | |
|----|---|
| 1 | × |
| 2 | × |
| 3 | √ |
| 4 | × |
| 5 | √ |
| 6 | × |
| 7 | × |
| 8 | √ |
| 9 | √ |
| 10 | × |
| 11 | × |
| 12 | √ |
| 13 | × |



邀请码: 12570104

APP首页右上角输入



该邀请码2026年08月26日前有效

计算机类2024级3班4班

课堂练习

邀请码: 97974770

APP首页右上角输入



课堂练习



该邀请码2026年08月26日前有效

计算机类2024级9班10班

地址传送指令

- 地址传送指令获取存储器操作数的地址
LEA r16/r32/r64,mem
#r16/r32/r64←mem的有效地址EA (不需类型一致)
#r16:实地址模式, r32: IA-32e , r64: 64位模式
- LEA指令类似的地址操作符OFFSET的作用
 - LEA指令在指令执行时计算出偏移地址
 - OFFSET操作符在汇编阶段取得变量的偏移地址
 - OFFSET无需在执行时计算、指令执行速度更快
 - LEA指令能获取汇编阶段无法确定的偏移地址

lea rsi, var[rip] #64位模式推荐

mov rdi, offset var #仅用于intel语法, 不稳定, 避免使用

地址传送程序



#数据段

dvar: .long 0x41424344

#代码段

mov eax,dvar[rip]

#EAX = 0x41424344

lea rsi,dvar[rip]

#RSI指向DVAR

mov ebx,[rsi]

#EBX = 0x41424344

lea rdi,dvar[rip]

#RDI指向DVAR

mov ecx,[rdi]

#ECX = 0x41424344

lea edx,[esi+edi*4+0x100] #EDX = ESI + EDI×4 + 0x100

〔例4-2〕：LEA指令实现算术运算

地址传送指令经常用于表达式求值。

```
/*eg0402.c*/  
int scale(int x, int y, int z){  
    long t=x+4*y+12*z;  
    return t;  
}
```

x: rcx
y: rdx
z: r8



```
lea    eax, [rcx+rdx*4]    #eax= x+4*y  
lea    edx, [r8+r8*2]      #edx=3*z  
lea    eax, [rax+rdx*4]    #eax=x+4*y+(3*z)*4
```

进栈指令PUSH



PUSH r16/m16/i16/seg

① $RSP = RSP - 2$ ② $SS:[RSP] = r16/m16/i16/seg$

PUSH r64/m64/i8/i32/i64

① $RSP = RSP - 8$ ② $SS:[RSP] = r64/m64/i8/i32/i64$

- 先将RSP减小作为当前栈顶
- 后将源操作数(立即数、通用寄存器和段寄存器内容或存储器操作数)传送到当前栈顶
- 以字或四字为单位操作
 - 进栈四字量数据时, RSP减8
 - 进栈字量数据时, RSP减2

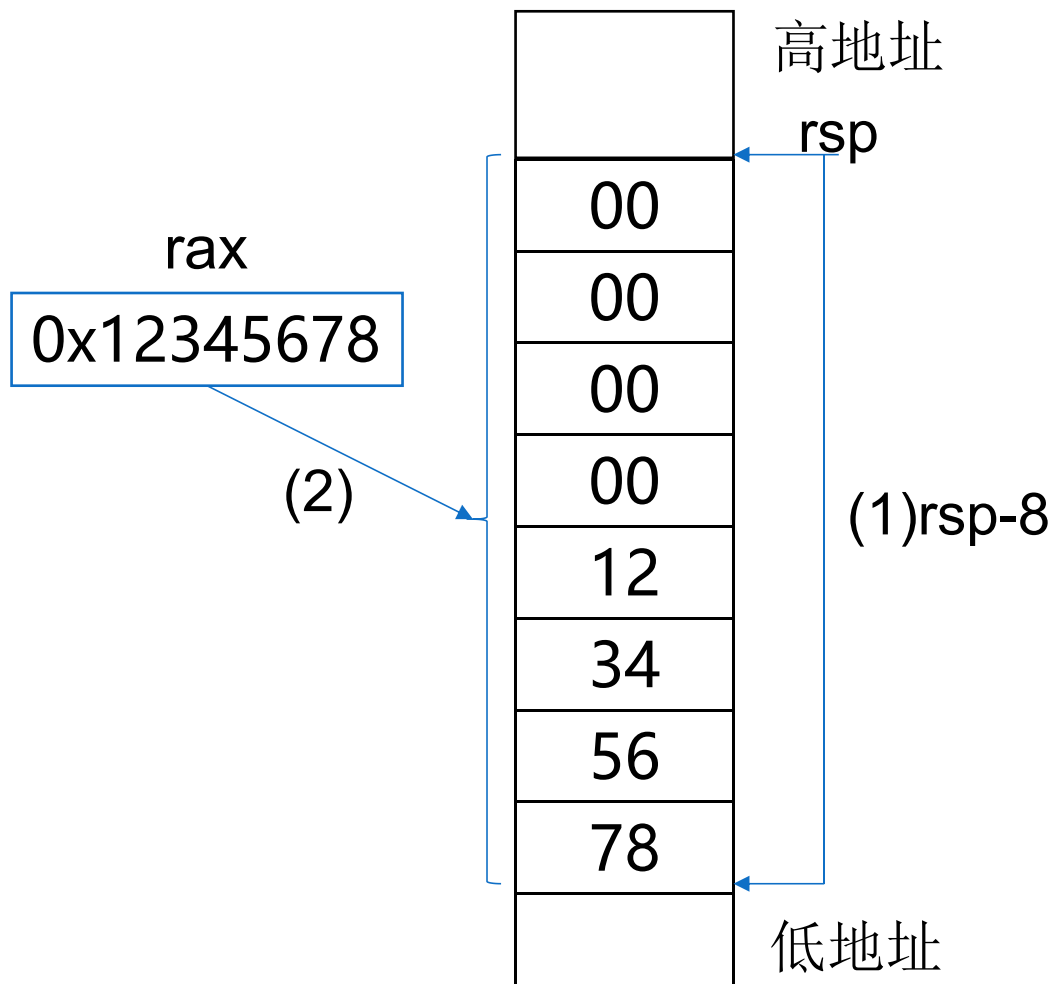


push rax

进栈操作



#进栈指令
push rax
#等同于如下两条指令
sub rsp, 8
mov [rsp], rax



出栈指令POP



POP r16/m16/seg

① r16/m16/seg = SS:[RSP] ② RSP = RSP + 2

POP r64/m64

① r64/m64 = SS:[RSP] ② RSP = RSP + 8

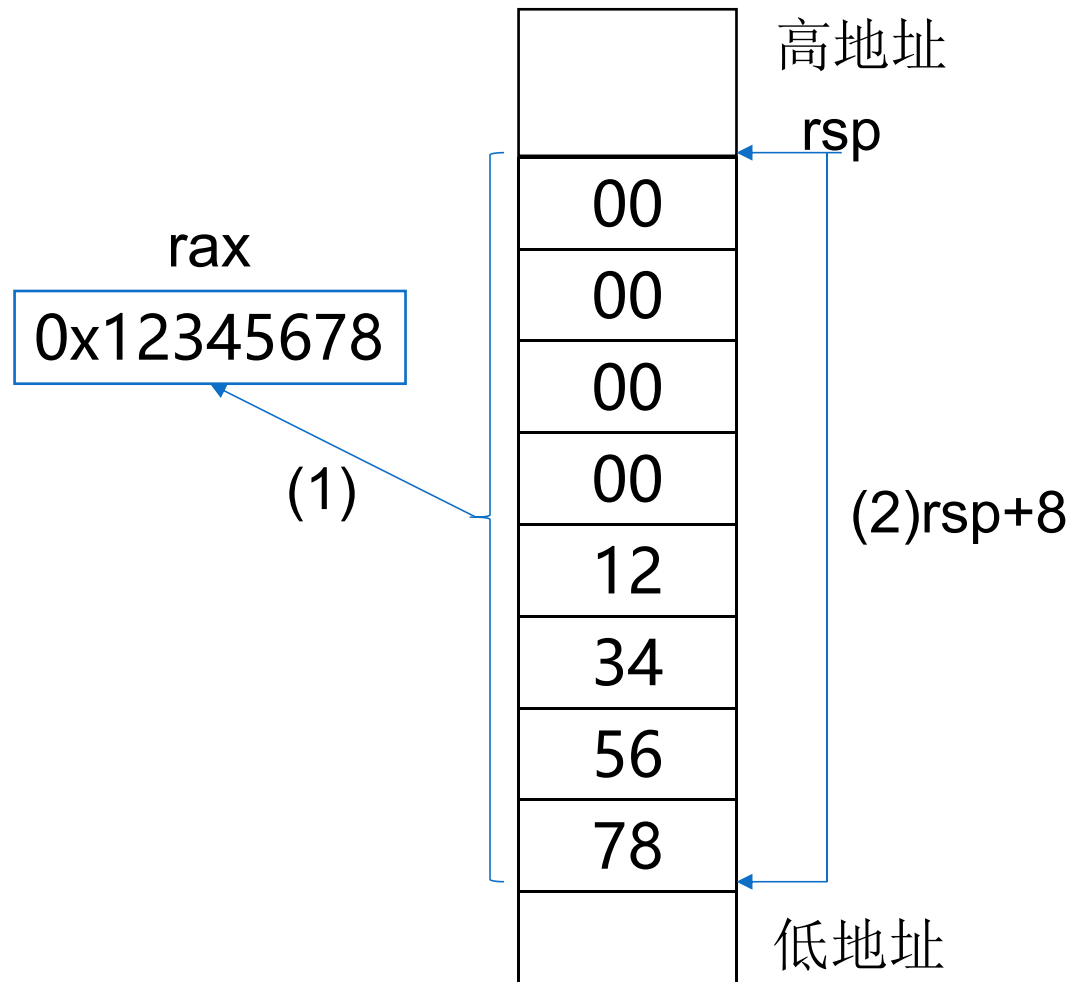
- 先将栈顶数据传送到目的操作数（通用寄存器、存储单元或段寄存器）
- 后ESP增加作为当前栈顶
- 以字或四字为单位操作
 - 出栈四字量数据时，RSP加8
 - 出栈字量数据时，RSP加2

pop rax

出栈操作



#进栈指令
pop rax
#等同于如下两条指令
mov rax, [rsp]
add rsp, 8



堆栈指令示例



#假设rsp= 0x00000052C01FFDE8

push rax	# rsp= 0x00000052C01FFDE0
pushf	# rsp=0x00000052C01FFDD8
mov rcx,[rsp]	# rcx=rflags=0x00000000000000202,
	# rsp=0x00000052C01FFDD8
mov rdx, [rsp+8]	# rdx=rax=0x0000000000000000A,
	# rsp=0x00000052C01FFDD8
popfq	# rsp=0x00000052C01FFDE0
pop rax	# rsp=0x00000052C01FFDE8

堆栈操作示例2



```
#数据段
.equ ten 10
dvar: .quad 0x67762000,0x12345678
#代码段
mov eax,dvar[rip+4]      #EAX = 0x12345678
push rax                #将RAX内容压入堆栈
push qword ptr ten      #将立即数以四字量压入堆栈
push dvar[rip]           #变量DVAR第一个数据入堆
pop rax                 #栈顶数据弹出到RAX
pop dvar[rip+4]          #栈顶数据弹出到DVAR + 4
mov rbx,dvar[rip+4]      #RBX = 0x0000000A
pop rcx                 #栈顶数据弹出到RCX
```

读写标志寄存器的值



RAX  **RFLAGS**

MOV RAX, RFLAGS
MOV RFLAGS, RAX

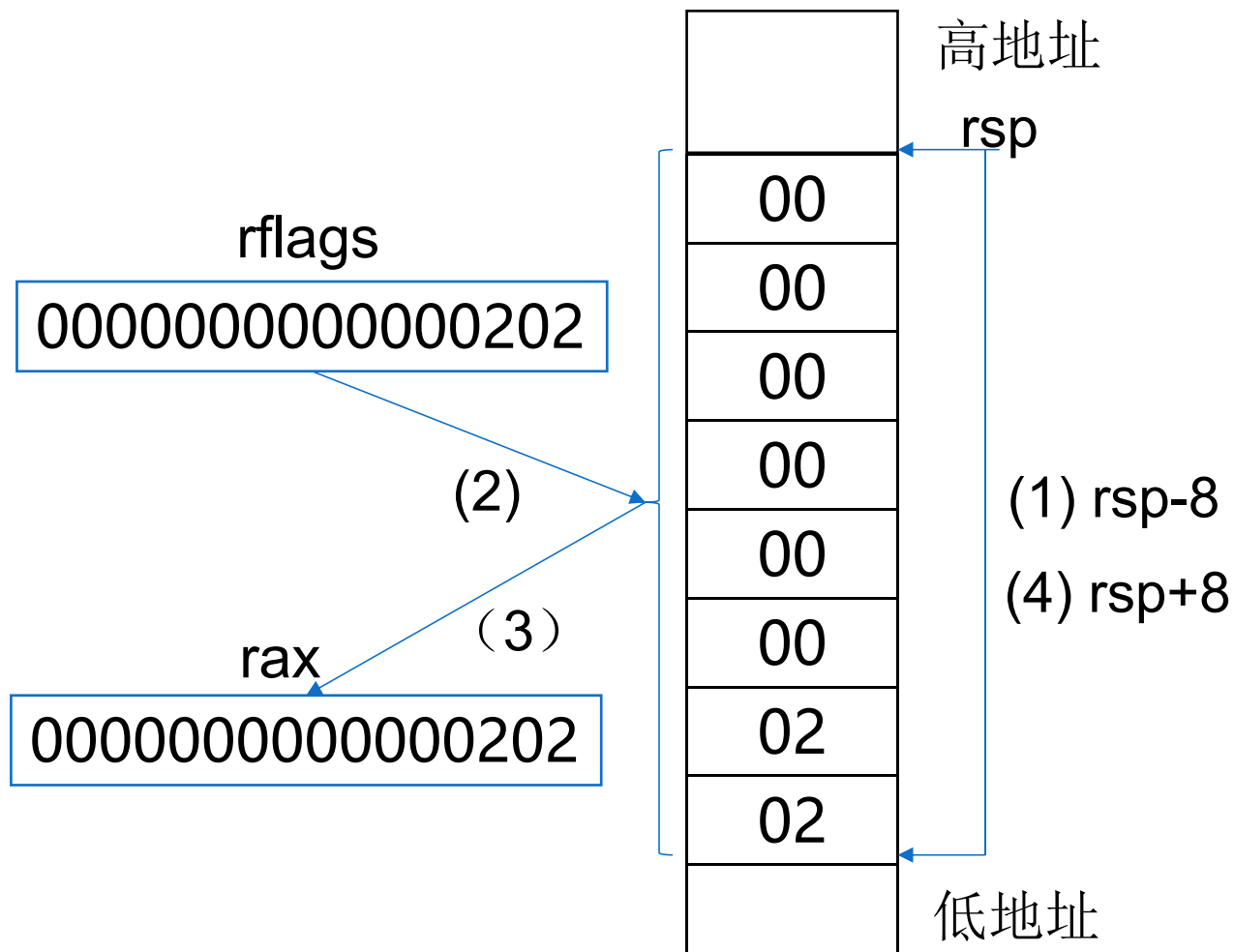


PUSH RAX
POPF

PUSHF
POP RAX

〔例4-3〕：查看标志寄存器的值

```
/*eg0403.S */  
.text  
pushf  
pop rax  
call disphx
```



练习



- 下面C语言语句如何用汇编指令实现?

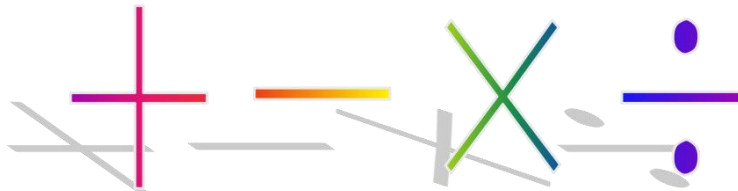
```
int x[10];  
x[0]=1;  
x[5]=x[0]+10;
```

- 下面汇编指令实现了什么功能?

```
lea edx, [rcx+rcx*4]  
lea edx, [rdx+rdx*2]
```


4.3 算术运算类指令

- 算术运算
 - 对数据进行加减乘除
 - 基本的数据处理方法
 - 加减运算有“和”或“差”的结果外，还有进借位、溢出等状态标志，也是结果的一部分
- 注意算术运算类指令对标志的影响
 - 掌握：加法和减法指令
 - 熟悉：乘法和除法指令
 - 理解：零位扩展和符号扩展



加法指令



- 加法指令 **ADD**
- 带进位加法指令 **ADC**

ADD/ADC reg,imm/reg/mem #reg←reg+imm/reg/mem(+CF)

ADD/ADC mem,imm/reg #mem←mem+imm/reg(+CF)

- 增量指令 **INC**

INC reg/mem #加1: reg/mem←reg/mem+1

- INC不影响进位标志CF
- 其他指令按定义影响全部状态标志位

按照运算结果相应设置各个状态标志为0或为1

数据传送类指令**不影响**（=不改变）状态标志
加法和减法指令**根据结果按定义改变**状态标志

〔例4-6〕：无符号64位二进制加法运算



```
.data
dvar1  .long 0x82347856, 0x67783000
dvar2  .long 0x12348998, 0x67762000
dvar3  .quad 0
.text
lea r8, dvar1[rip]
lea r9, dvar2[rip]
lea r10, dvar3[rip]
```

〔例4-6〕：无符号64位二进制加法运算



```
mov eax,[r8]
add eax,[r9] #低位双字相加
              #eax=0x946901ee,EFLAGS=0x00000286
mov [r10], eax
mov eax,[r8+4]
adc eax,[r9+4] #高位双字相加, 再加CF,
              #eax=0xceee5000,EFLAGS=00000A86
mov [r10+4], eax

mov rax,dvar3[rip]
call disphx
```

〔例4-7〕：修改例4-6的代码段



..... #与例4-6相同

.text

mov rbx,0

.....

mov eax,[r8+rbx*4]

add eax,[r9+rbx*4]

#低位双字相加,

#eax=0x946901ee,EFLAGS=0x00000286

mov [r10+rbx*4],eax

inc rbx

mov eax,[r8+rbx*4]

adc eax,[r9+rbx*4]

#高位双字相加, 再加CF,

#eax=0xceee5000,EFLAGS=00000A86

mov [r10+rbx*4],eax

mov rax,dvar3[rip]

减法指令



- 减法指令 SUB

- 带借位减法指令 SBB

SUB/SBB reg,imm/reg/mem #reg←reg-imm/reg/mem(-CF)

SUB/SBBmem,imm/reg #mem←mem-imm/reg(-CF)

- 减量指令 DEC

DEC reg/mem # reg/mem←reg/mem-1

- 求补指令 NEG

NEG reg/mem #reg/mem←0—reg/mem

- 比较指令 CMP

SUB reg,imm/reg/mem; reg-imm/reg/mem

SUB mem,imm/reg; mem-imm/reg

- 除DEC不影响CF标志外
- 其他按定义影响全部状态标志位

〔例4-10〕：64位二进制减法运算



```
.data
dvar1: .long 0x82347856,0x67783000
dvar2: .long 0x12348998,0x67762000
dvar3: .quad 0
.text
mov rbx,0
lea r8, dvar1[rip]
lea r9, dvar2[rip]
lea r10, dvar3[rip]
```

〔例4-10〕：64位二进制减法运算



```
mov eax,[r8]
```

```
sub eax,[r9]
```

#低位双字相减

#eax=0x6fffeebe,EFLAGS=0x00000A16

```
mov [r10],eax
```

```
mov eax,[r8+4]
```

```
sbb eax,[r9+4]
```

#高位双字相减，再减CF，

#eax=0x00021000,EFLAGS=0x00000206

```
mov [r10+4],eax
```

```
mov rax,dvar3[rip]
```

.....

大小写字母转换



大写=小写-0x20
小写=大写+0x20

mov al, 'A' #al=0x41 ('A' 的ASCII码)

add al, 0x20 #al=0x61 ('a' 的ASCII码)

mov al, 'a' #al=0x61 ('a' 的ASCII码)

sub al, 0x20 #al=0x41 ('A' 的ASCII码)

求补指令NEG



- 可用于对负数求补码或由补码求其绝对值

mov ax,0xff64

neg al #AX=0xFF9C, OF=0, SF=1, ZF=0, PF=1, CF=1

sub al,0x9d #AX=0xFFFF, OF=0, SF=1, ZF=0, PF=1, CF=1

neg ax #AX=0x0001, OF=0, SF=0, ZF=0, PF=0, CF=1

dec al #AX=0x0000, OF=0, SF=0, ZF=1, PF=1, CF=1

neg ax #AX=0x0000, OF=0, SF=0, ZF=1, PF=1, CF=0

乘法指令



- 无符号数乘法指令 **MUL**
- 有符号数乘法指令 **IMUL**
- 计算二进制数乘法: $0xA5 \times 0x64$

- 用MUL指令作无符号数乘法:

$$0x4074 \text{ (} = 16500 \text{)} = 0xA5 \text{ (} = 165 \text{)} \times 0x64 \text{ (} = 100 \text{)}$$

- 用IMUL指令作有符号数乘法:

$$0xDC74 \text{ (} = -9100 \text{)} = 0xA5 \text{ (} = -91 \text{)} \times 0x64 \text{ (} = 100 \text{)}$$

乘法指令



指令类型	操作数组合及功能	举例
无符号数乘法 MUL src	$AX = AL \times r8/m8$ $DX.AX = AX \times r16/m16$ $EDX.EAX = EAX \times r32/m32$ $RDX.RAX = RAX \times r64/m64$	mul bl imul bx mul dword ptr var[rip] imul dword ptr var[rip] mul qword ptr var[rip] imul qword ptr var[rip]
有符号数乘法 IMUL src		
双操作数乘法 IMUL dest,src	$r16 = r16 \times r16/m16/i8/i16$ $r32 = r32 \times r32/m32/i8/i32$ $r64 = r64 \times r64/m64/i8/i64$	imul eax,10 imul ebx,ecx imul rbx,rcx
三操作数乘法 IMUL dest,src,imm	$r16 = r16/m16 \times i8/i16$ $r32 = r32/m32 \times i8/i32$ $r32 = r64 \times r64/m64/i8/i64$	imul ax,bx,-2 imul eax,dword ptr [rsi+8],5 imul rax,qword ptr [rsi+8],5

- 无符号除法指令 DIV
- 有符号除法指令 IDIV
- 除法指令可能产生除法溢出
 - 对DIV指令，除数为0，或者在字节除时商超过8位，在字除时商超过16位，或者双字除时超过32位，则发生除法溢出
 - 对IDIV指令，除数为0，或者在字节除时商不在 - 128 ~ 127 范围内，在字除时商不在 - 32768 ~ 32767 范围内，或者在双字除时商不在 - 2^{31} ~ $2^{31} - 1$ 范围内，则发生除法溢出
- 除法错溢出，将产生编号为0的内部中断

除法指令



指令	操作数组合及功能	举例
无符号除法: <code>DIV src</code>	$AL \leftarrow AX \div r8/m8$ 的商 $AH \leftarrow AX \div r8/m8$ 的余数 $AX \leftarrow DX$. $AX \div r16/m16$ 的商	<code>div bl</code> <code>idiv bl</code> <code>div bx</code>
有符号除法: <code>IDIV src</code>	$DX \leftarrow DX$. $AX \div r16/m16$ 的余数 $EAX \leftarrow EDX$. $EAX \div r32/m32$ 的商 $EDX \leftarrow EDX$. $EAX \div r32/m32$ 的余数 $RAX \leftarrow RDX$. $RAX \div r64/m64$ 的商 $RDX \leftarrow RDX$. $RAX \div r64/m64$ 的余数	<code>idiv bx</code> <code>div ebx</code> <code>idiv ebx</code> <code>div rbx</code> <code>idiv rbx</code>

〔例4-14a〕：无符号数除法



```
void eg0414a(unsigned long x, unsigned long y,  
unsigned long *qp, unsigned long *rp)  
{  
    unsigned long q=x/y;  
    unsigned long r=x%y;  
    *qp=q;  
    *rp=r;}
```



```
# ecx=x,  edx=y,  r8=qp,  r9=rp  
mov  eax, ecx  
mov  r10d, edx          #y保存到r10d, 为什么?  
mov  edx, 0             #被除数edx.eax  
div  r10d  
mov  DWORD PTR [r8], eax #eax=商  
mov  DWORD PTR [r9], edx #edx=余数
```

〔例4-14b〕：有符号数除法



```
void eg0414b(long x, long y, long *qp, long *rp){  
    long q=x/y;  
    long r=x%y;  
    *qp=q;  
    *rp=r;}  
↓
```

```
# ecx=x,  edx=y,  r8=qp,  r9=rp  
mov  eax, ecx  
mov  r10d, edx          #y保存到r10d  
cdq                          # eax符号扩展到edx.eax  
idiv  r10d  
mov  DWORD PTR [r8], eax  #eax=商  
mov  DWORD PTR [r9], edx  #edx=余数
```


零位扩展和符号扩展指令

- 零位扩展对应无符号数：MOVZX指令
 - 前面加0实现位数扩展
 - 0x80：8位无符号数，零位扩展为16位：0x0080
- 符号扩展对应符号数：MOVSX指令
 - 前面加符号位（最高位）实现位数扩展
 - 0x64：8位有符号数，符号扩展成16位：0x0064
 - 0xFF00：16位有符号数据
 - 符号扩展成32位：0xFFFFFFFF00，都表达真值：-256
 - 真值-1，字节量补码：0xFF
 - 字量补码：0xFFFF，双字量补码：0xFFFFFFFF

 位数加长，大小没变

零位扩展和符号扩展指令



指令类型	指令	举例
零位扩展	MOVZX r16,r8/m8	movzx di,bvar[rip]
	MOVZX r32,r8/m8/r16/m16	movzx eax,ax
	MOVZX r64,r8/m8/r16/m16	movzx rax,bl
符号扩展	MOVSX r16,r8/m8	movsx ax,al
	MOVSX r32,r8/m8/r16/m16	movsx edx,bx
	MOVSX r64,r8/m8/r16/m16	movsx rax,bl
	MOVSXD r64, r32/m32	movsxd rax, ebx

零位扩展和符号扩展指令示例



mov al,0x82	/*al=0x82*/
movzx bx,al	/*bx=0x0082*/
movzx ebx,al	/*ebx=0x00000082*/
mov cx, 0x1000	/*cx=0x1000*/
movzx edx,cx	/*edx=0x00001000*/

mov al,0x82	/*al=0x82*/
movsx bx,al	/*bx=0xFF82*/
movsx ebx,al	/*ebx=0xFFFFFFFF82*/
mov cx,0x1000	/*cx=0x1000*/
movsx edx,cx	/*edx=0x00001000*/

除法溢出



- 除法指令可能产生除法溢出
 - 对DIV指令，除数为0，或者在字节除时商超过8位，在字除时商超过16位，在双字除时超过32位，或四字除时超过64位则发生除法溢出。
 - 对IDIV指令，除数为0，或者在字节除时商不在 - 128 ~ 127 范围内，在字除时商不在 - 32768 ~ 32767 范围内，在双字除时商不在 - $2^{31} \sim 2^{31} - 1$ 范围内，或者四字除时商不在 - $2^{63} \sim 2^{63} - 1$ 范围内则发生除法溢出。
- 除法错溢出，将产生编号为0的内部中断

```
mov ax, 20000
```

```
mov bl, 10
```

```
div bl      #20000 ÷ 10 = 2000, 商在AL中放不下, 产生溢出
```

```
#用16位除法避免发生溢出:
```

```
mov dx, 0
```

```
mov ax, 20000
```

```
mov bx, 10
```

```
div bx      #20000 ÷ 10 = 2000, 商在AX中可以放下, 不产生溢出
```

下面指令执行后**eax**中的值是什么？用公式表示

```
movsx eax, cx  
imul eax, 9  
cdq  
mov ebx, 5  
idiv ebx  
add eax, 32
```



邀请码: 12570104

APP首页右上角输入



该邀请码2026年08月26日前有效

计算机类2024级3班4班

课堂练习

邀请码: 97974770

APP首页右上角输入



课堂练习



该邀请码2026年08月26日前有效

计算机类2024级9班10班

第6讲教学内容



- 阅读教材
 - 教材章节：第4.4节
- 点播视频
 - 慕课章节：6
- 自测练习
 - 慕课章节：单元测试6

教学要点

- ✓ 逻辑运算类指令
- ✓ 移位指令

教材第3.3节： （常用的） 位操作类指令



AND、OR、XOR和NOT指令	逻辑运算
SHL/SAL、SHR和SAR指令	移位
ROL、ROR、RCL和RCR指令	循环移位

设置CF = OF = 0
影响SF, ZF和PF

逻辑运算类指令

AND reg,imm/reg/mem #reg←reg $\wedge$ imm/reg/mem

AND mem,imm/reg #mem←mem $\wedge$ imm/reg

OR reg,imm/reg/mem #reg←reg $\vee$ imm/reg/mem

OR mem,imm/reg #mem←mem $\vee$ imm/reg

XOR reg,imm/reg/mem #reg←reg $\oplus$ imm/reg/mem

XOR mem,imm/reg #mem←mem $\oplus$ imm/reg

NOT reg/mem #reg/mem← $\sim$ reg/mem

不影响状态标志位

逻辑运算指令有什么作用?